

mov ax, bx
↓
destination → source

day / date:

CHAP #1

LECTURE - 1

Tuesday
19/8/25.

⇒ Buses: Set of wires that connects with memory.

- Address Bus: Send address of cell to processor & its uni-directional, to read & write. (→)
- Data Bus: Processor can read & write from & to memory. Bidirectional: ↔
- Control Bus: Control signals sent by processor. Bidirectional: ↔

LECTURE - 2

Thursday
21/8/25.

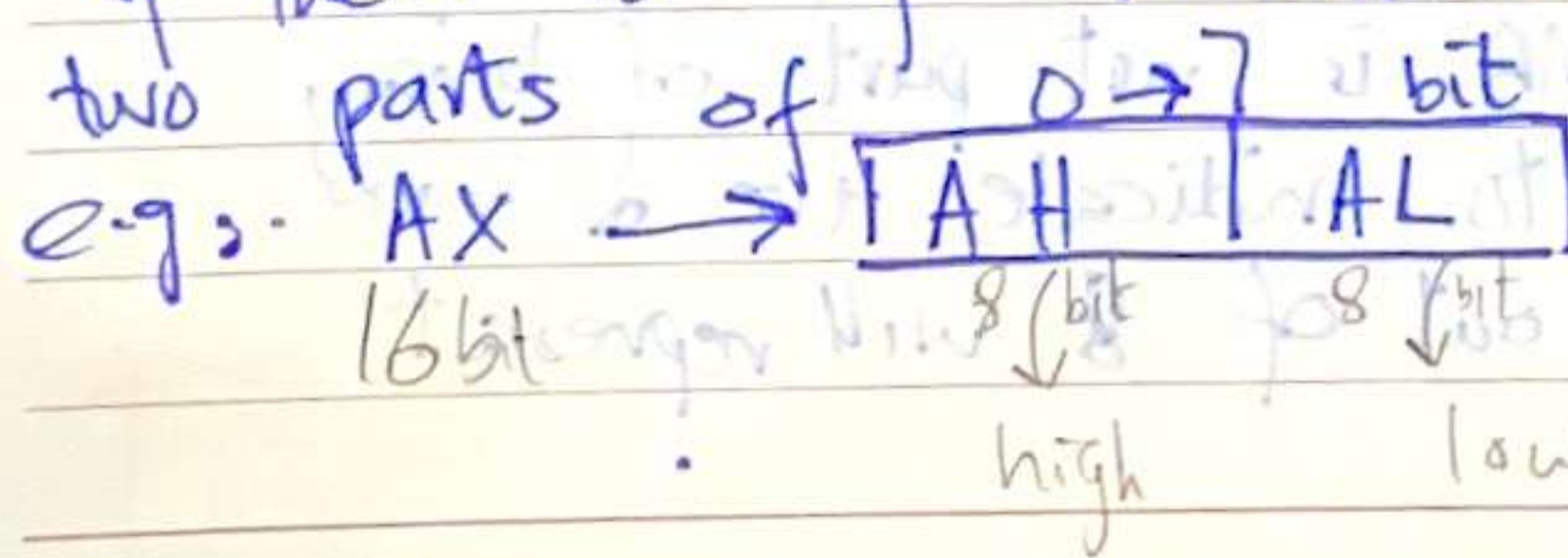
Intel iAPX88 Architecture:

- 16 Bit processor
- 8 Bit data bus
- 20 bit Address Bus
- $2^{20} = 1 \text{ MB}$ RAM
- 14 Registers of 16 bits each

Registers: Temporary memory unit.

⇒ General Registers in Intel 8088:

AX, BX, CX and DX are general reg. Each of them is of 16 bits but each is divided in two parts of 8 bit & 8 bit.



15	8	7	0
AH	AL		
BH	BL		
CH	CL		
DH	DL		

SI } Index Registers
DI }
SP } pointer registers
BP } stack related

⇒ Flag Register:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
				0	D	I	T	S			A		P		C

- bits:
- 0) Carry (C): When two 16-bit numbers added & result is in 17 bits, the extra bit is stored here.
- For borrow as well.
 - Unsigned bits.

- 2) Parity (P): Number of one bits in binary number used to check data integrity. could be odd or even.

- 3) Auxiliary (A): This is carry flag for 4 bit system (nibble) which could be used in BCD addition or subtraction.

- 7) Sign Flag: A signed number is whose LMB will show the sign of number. ~~1 = negative~~ 1 = negative 0 = positive
- A signed number needs a 2's complement if LMB=1 and we need to find the original number. LMB is the part of number

∴ The LMB here will be stored in sign flag.

- ★ Signed Magnitude number: LMB is not part of binary number and it is just to indicate the +ve or -ve. In this case, only 7 bits out of 8 will represent range.

- 8) Trap (T): Role in Debugging.

- 9) Interrupt (I): When processor is interrupted with a higher priority task.

- 10) Direction (D): For string instruction. Tells memory that whether the operation has to be done from bottom → top (D=0) or vice versa (D=1).

- 11) Overflow (O): Carry flag for signed number.

Unsigned Numbers (4 bit): 0 → 15

Signed Numbers (4-bit): -8 → +7

signed mag. Number (4 bit): -7 → +7.

⇒ Adding 3 constants:

LECTURE - 3

Thursday
26/8/25

Original instructions that 100 after it is hexadecimal from memory

mov ax, 5 → more 5 to ax register. ax = 0x0005

mov bx, 10 → bx = 0x000A

$$0x = 5 + A = 1F$$

add ax, 13 OR mov bx, 13 → Always reuse registers.

add ax, bx

$$ax \leftarrow 15 + 13 = 28 \rightarrow 1C$$

mov ax, 0x100
int 0x21

Terminating instruction.

day / date:

test.asm → Name of file on notepad.

asm → Assembler
afd → debugger (Shows value of registers)

asm test.asm -o test.com → Command in DosBox.

Sub ax, bx → ax = ax - bx.

translate assembly to machine code

* 'Mov' instruction does not effect flag register.

* 'Add' and 'Sub' instructions many trigger: overflow, sign, zero, carry, auxiliary, parity flags.

line no.	offset in hexadeciml	Machine code in hexa. Assembly code
1		
2	00000000	BB0500
3		
4		
5		
6		

→ Data label = variable

AB: nasm → Assembler, afd → debugger

① test.asm : Make this in Notepad

② nasm test.asm -o test.com -b test.lst } Type these two commands in dosbox command prompt to connect Notepad file with it.

③ Afd test.com

D:\University\Sem3\Cal lab\Assembly\Dosbox\Porta\Data\settings\ dosbox

* Editing the 2nd line of it and pasting the above

need to point our dosbox towards the location of new project.

* Typing ② & ③ below the dosbox notepad file will directly open our project once we open app.



VERY IMP. "mov ax, 5" = "mov ax, 0x0005" = "BB 05 00"

Listing File:

Line no.	offset in Hexa (IP register)	Machine Code in Hexa:	Assembly Code:
1	00000000	BB 05 00	mov ax, 5
2			
3	00000003	BB 0A 00	mov bx, 10
4	00000006	01 D8	add ax, bx
5	00000008 (6+2)	BB 0F 00	mov bx, 15
6	0000000B	01 D8	add ax, bx
7	0000000D	B8 00 4C	mov ax, 0x4C00
8	00000010	CD 21	int 0x21

→ Size of Exe file:

Formula: (last offset)₁₆ + 2 = 18 bytes (add all the bytes of)

⇒ Little Endian Form: least significant bit (usually at right) is stored from left of memory and most signif. bit usually stored at left is stored at right

e.g: 0005 → 05 00

16bit: LSB is stored first & MSB is stored after this



Sub ax, bx → ax = ax - bx

* 'mov' instruction doesn't effect flag register
 * 'Add' and 'Sub' instruction may trigger: overflow, sign, zero, carry, auxiliary, parity flags.

* Variable is "Data Label"

⇒ Creating Memory cell in memory:

num 1: db 5 → initialising value in variable.

Variable name define 1 Byte

num 2: dw 7 → value in num 2

define 2 Bytes (word)

"data"

for targeting variables/arrays we created ourselves

Read mov ax, [num 2] : transfer data from memory

(i.e. num 2 variable) to processor (ax).

mov bx, num 2 : transfer address of num 2 to bx (bx points num 2)

Write: mov [num 1], ax : transfer data from processor (ax)

to memory (num 1 variable).

→ Define Array:

arr 1: dw 5, 10, 15

0x0005 0x000A 0x000F

Contexts of array

Arr 1		
A0	05 (LSB)	1st Index
A1	0A (MSB)	
A2	0A	2nd Index
A3	00	
A4	0F	3rd Index
A5	00	

arr 2: db 5, 10, 15

arr 2		
B0	05	
B1	0A	
B2	0F	

→ Accessing Element of 1st Byte arr 1:

First cell's (of array) address

mov ax, [arr 1] // ax = 0005

mov bx, [arr 1 + 2] // bx = 000A

Third Cell: since each index is taking 2 bytes as 'dw'.

mov ax, [arr 1 + 1] // ax = 0A00 (little endian)

mov al, [arr 1] // al = 05 (Right most → LSB into al)

mov ax, [arr 2] // ax = 0A05 since arr 2 is 1 byte

array and ax is 2 byte register ∴ 2 values in little endian form will be transferred to ax

mov al, [arr 1 + 2] // al = 0A

mov ax, [arr 2 + 2] // ax = garbage value 0F

→ IF Hexa values stored in Array:

arr 3: dw 0x1234

arr 3		
C0	34	LSB
C1	12	MSB
C2		

* You cannot reference/register like: [ax] ✓

* AX (accumulator reg.) is only one that cannot be referenced.

Sub ax, bx → ax = ax - bx

* 'Mov' instruction doesn't effect flag register
 * 'Add' and 'Sub' instruction may trigger: overflow, sign, zero, carry, auxiliary, parity flags.

LECTURE-4 | Thursday 28/8/25.

* Variable is "Data Label"

⇒ Creating Memory cell in memory:

num 1: db 5 → initialising value in variable.

Variable name define 1 Byte
 num 2: dw 7 → value in num 2

define 2 Bytes (word)

for targeting variables we created ourselves

load mov ax, [num 2] : transfer data from memory (i.e. num 2 variable) to processor (ax).

mov bx, num 2 : transfer address of num 2 to bx (bx point num 2)

write: mov [num 1], ax : transfer data from processor (ax) to memory (num 1 variable).

→ Define Array:

arr 1: dw 5, 10, 15

Each element is taking 2 cells (2 Bytes)

Contexts of array	
A0	05 (LSB)
A1	00 (MSB)
A2	0A
A3	00
A4	0F
A5	00

1st Index
2nd Index
3rd Index

arr 2: db 5, 10, 15

B0	05
B1	0A
B2	0F

→ Accessing Element of 1st byte arr 1:

mov ax, [arr 1] // ax = 0005
 mov bx, [arr 1 + 2] // bx = 000A

Third Cell: since each index is taking 2 bytes as 'dw'.

mov ax, [arr 1 + 1] // ax = 0A00 (little endian)

mov al, [arr 1] // al = 05 (Right most → LSB into al)

mov ax, [arr 2] // ax = 0A05 since arr 2 is 1 byte

array and ax is 2 byte register ∴ 2 values in little endian form will be transferred to ax

mov al, [arr 1 + 2] // al = 0A

mov ax, [arr 2 + 2] // ax = garbage value 0F

→ IF Hexa values stored in Array:

arr 3: dw 0x1234

C0	34	LSB
C1	12	MSB
C2		

* You cannot reference/register (like: [ax])

* AX (accumulator reg.) is only one that cannot be referenced.

Size Definition is needed when any register is not involved in the statement (means no size is allocated).

→ Withing constant into memory.

`mov byte [num1], 5` // num1 = 5

`mov word [num2], 0x10AF` // num2 = 10.

`mov [num2], [num1] X ILLEGAL.`

Q: Store the sum of numbers on first 3 index and store the result in 4th index of array.

arr1: dw 5, 10, 15, 0

Answer: 1 `org x100`

2 `mov ax, [arr1]` // ax = 5

3 `mov bx, [arr1+2]` // bx = 10

4 `add ax, bx` // ax = 5 + 10 = 15

5 `mov bx, [arr1+4]` // bx = 15

6 `add ax, bx` // ax = 15 + 15 = 30

7 `mov [arr1+6], ax` // arr[4] = 30

8 `mov ax, 0x4C00`

9 `int 0x21`

10 `arr1: dw 5, 10, 15, 0`

// defined array after the code since doing it right after [org x100] will disturb memory sequence of 0x100.

→ Listing File of Previous Question Array.

Offsets	Machine Code
1	
2	00000000
3	A1 [17 00]
4	OP Code for moving arr's data in ax register
5	
6	
7	
8	
9	
10	00000017
	0500 0400
	0F00, 0000
	arr1: dw 5, 10, 15, 0

* It could be 'add' or 'sub' etc instead of 'mov'.

1) Direct Addressing: `mov ax, [num1]` values from these addresses will directly go to ax.

2) Base Register "indirect" (bx, bp): `mov bx, num1` OR `mov ax, [bx]`

3) Index Register "indirect" (si, di): `mov si, num1` OR `mov ax, [si]`

NOTE: "mov bx, [num + dx - 3]" adding or subtracting any register with memory (num) inside offset is illegal.

• The 7 ways described inside [] are the only ways to access the memory, else it will be illegal. Main goal is to access 'ax' using different methods.

day / date:

11

OR with bp.

Get a then value in Profit

$$S(a_1): \text{join } ax.$$

with du

of one inventory.

No two ba

$$e.g (bx + bp) \times$$

1

1. The first part of the text discusses the importance of maintaining accurate records of all transactions, including sales, purchases, and expenses. It emphasizes that proper record-keeping is essential for determining the correct amount of tax liability and for defending against potential audits.

① ② ③ ④ ⑤ ⑥ ⑦ ⑧ ⑨ ⑩ ⑪ ⑫ ⑬ ⑭ ⑮ ⑯ ⑰ ⑱ ⑲ ⑳ ㉑ ㉒ ㉓ ㉔ ㉕ ㉖ ㉗ ㉘ ㉙ ㉚ ㉛ ㉜ ㉝ ㉞ ㉟ ㊱ ㊲ ㊳ ㊴ ㊵ ㊶ ㊷ ㊸ ㊹ ㊺ ㊻ ㊼ ㊽ ㊾ ㊿

• which we give

三

10

rough which it can

as physical address

KACH
www.kach

For our understanding

1

de fait

segments

~~These~~

di

after segment AB.

1. $\frac{1}{2}$ → $\frac{1}{2}$

U.S. No. 100

Savore

bracelet

is also

email address

memory.

PIX requires LE_7 -ex, s

for segment

default segment

1890, May 11

1995

from seq. regist

7 appen

DDDO

$$\begin{array}{r} 20010 \\ 70010 \\ \hline 27020 \end{array}$$

DEDO

25

→ physics

KAGHA
www.kagha.com

Do
Kauz
www.kauz.it
adare

LECTURE - 6

Thus day 4/9/25

bx = 0x9100

mov ax, [bx + 0x7000]

EFFECTIVE ADDRESS

9100
+ 7000

10100
↓
*Segment Wrap Around: When offset (10100) gets more than 4 bits so it becomes 0x0100 which is smaller than inputs.

d: bx = 0x0100

ds = 0xFFFF. (segment Number)

mov [bx + 0x0100], ax

0100

0200 → offset in limit of 4 bits ✓

* Now we'll append a '0' in ds register & add it in offset (0x0200) to physical address.

0xFFFF0 → appending 0's → 0xFFFF00

0xFFFF00

0200

Physical Memory: When physical memory exceeds 20 bits

ignore 62 bit since it goes above 20 bit (i.e. 5 bits)

mov ax, [bx + si]: Target's data segment since both 'bx' and 'si' are from ds.

mov ax, [bp + si]: Even tho 'bp' & 'si' have different segment groups, we'll prioritise the segment group of base register i.e. bp → stack segment

JUMP INSTRUCTION

CHAP # 3

⇒ Unconditional: jmp destination address

* we can name entire line instruction into a label name

jmp label1

label1: mov ax, bx

* we use to declare variables/arrays at the end of code. Now we can use jump instruction & write all variables & declaration in start.

[org 0x100]

jmp start

num 1: db 1, 2, 3

start

* "cmp" should be right above the jump statement since jmp statement works on register's values & if there is any statement b/w cmp & jmp like add, sub etc, it may affect the registers hence the jmp statement

⇒ Conditional Jump: **je** destination

cmp compare instruction: For a comparison only. It won't change any register value

[WORKING]: 2

cmp ax, bx: Subtract values of ax from bx & update flag registers: OF, SF, CF, ZF, PF, AF.

OR **cmp ax, [num1]** size definition

OR

cmp ax, 5 Needed when register is not involved in source

OR

cmp word/byte [num1], 5 num1 ki byte compare krni hai

cmp ax, 5
je label
If $ax=5$, we will jump to label.

label: _____
* 'je' is internally checking zero flag. If $zero\ flag == 1$, then true comp.

⇒ Types of conditional jumps:

- je**: jump if equal
- jz**: jump if zero
- jne**: jump if not equal
- jnz**: jump if not zero

* rest are in Book

⇒ Unsigned comparison:

ja: jump if above ($n1 > n2$) → checks carry flag

⇒ Signed comparison:

jg: jump if greater

mov al, 0xFF → Unsigned: 1111 1111 = **255**
mov ax, 1 → Signed: 1111 1111
cmp ax, al $1 < 255$ → **Signed: 1111 1111 + 65536 = 65537**
ja label1 $255 > ax(1)$: TRUE → **0000 0001 → -1**
jg label2 $-1 > ax(1)$: FALSE

LECTURE - 7 Tuesday
9/9/25

CONDITIONAL STATEMENTS:

Q: if $ax=0$
je next
else **bx=1** when true **mov bx, 2**
jmp end // so 'next' isn't implemented
next: mov bx, 1 // if bx is already equal to 2.
end: mov ax, 0x4C00
int: 0x21

Q: if ($ax=10$ and $bx=1$):

$CX=1$

cmp ax, 10
jne end
cmp bx, 1
jne end
mov cx, 12
end: mov ax, 0x4C00
0x21
 $P1 = 3 + 21$

Formula from Signed to Unsigned (Vice versa)

Signed value = Unsigned - 65536

LOOP

day / date:

Q: arr1: dw 10, 20, 30, 40, 50, 60, 70, 80, 90, 100
total: dw

Add all elements of arr1 and store result in total using loop.

1 long ox1001
2 jmp start

3 arr1: dw 10, 20, 30, 40, 50, 60, 70, 80, 90, 100

4 total: dw 0

5 start: mov ax, 0

6 mov bx, 0

7 L1: add ax, [arr1 + bx]

8 add bx, 2 // move to next index since 2 byte array

9 cmp bx, 20 // checking end of array has reached.

10 jne L1 // 20, because $2 \times 10 = 20$ bytes

11 mov [total], ax

12 mov ax, 0x4C00

13 int 0x21

1 00000000 EQ 1600

2 0000 0003

5 0000 0019 B8 0000

1600 = (16)10 : This is distance from jump statement to label i.e. start.

16 + 3 = 19
EQ 1600 occupies 3 bytes.

* The above jump was "long jump" which have 2 bytes (16 00).

* Short Jump: we can jump only 1 byte.

Means we can jump -128 bits previous and +127 bit forward.

Offset	Machine	Assembly
7 0000 001F	code jne	L1: add ax, [arr1 + bx]
10 0000 002B	75 F2	jne L1
11 0000 002D		mov [total], ax

2D instead of 2B as the 'IP' register already moved to next instr. after executing for that we take 2's complement of DE which is 'F2'.

Shifting:

⇒ Shift Right:

Used for dividing an unsigned number by any number of 2^n ($n \in \mathbb{Z}$).

Eg: 0100(4) right shift 4 → 0010(2)

12 ÷ 4 → 1100 ÷ 2² 2 shifts → 0011

1 ÷ 2 → 0001 ÷ 2¹ 1 shift → 0000

LECTURE - 8

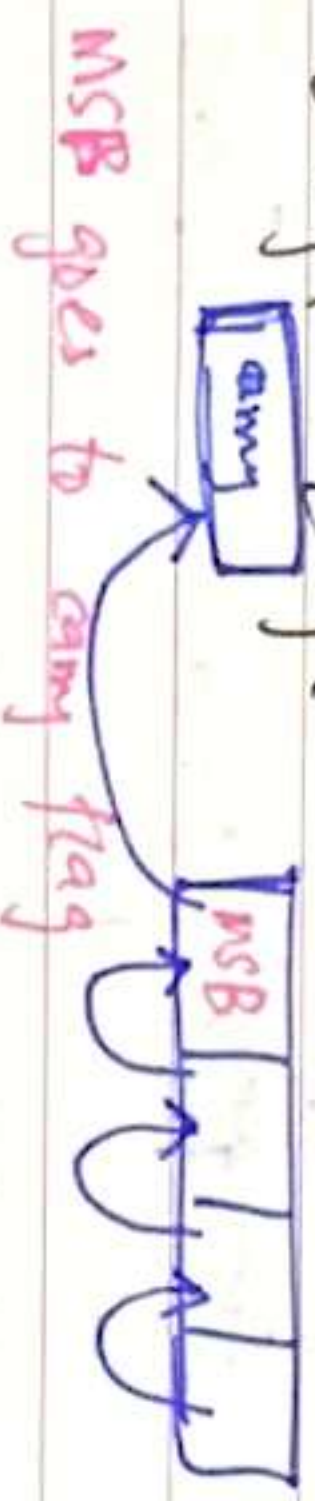
Thursday 11/9/25

shift right

shr ax, 2

shifts the values in ax reg. 2 times.

⇒ Shift Left:

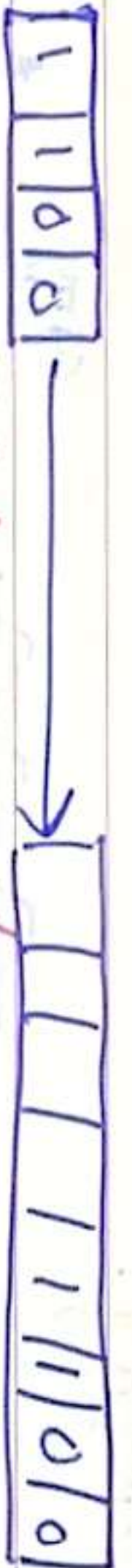


For multiplication with 2^n ($n \in \mathbb{Z}$) $\rightarrow n = \text{no. of left shifts}$.

e.g: $6 \times 2 = 0110 \times 2$
 $\xrightarrow{\text{left shift}} 1100$ (i.e.: 12)

[IMP]: $12 \times 2 = 24$
 $1100 \rightarrow 1000$

∴ if MSB of any binary number is '1', then we'll use double sized register so the overflow bit from left goes to 5th cell.



new shifting to left: will be accommodated as answer = 11000 (which is 24 i.e. 12×2).

Shift left

shl ax, 1

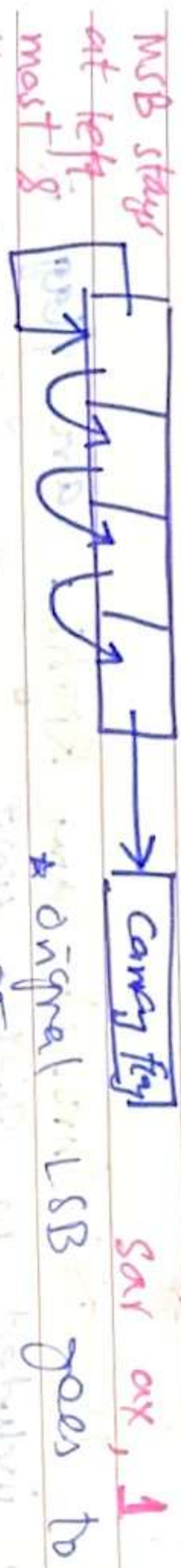
OR Same

sal ax, 1

Shift arithmetic left

Signed division by 2^n

⇒ Shift Arithmetic Right: To preserve the MSB which is signed bit & it won't affect the sign of number.



e.g: $1100 \xrightarrow{\text{goes to carry}} 1110$

-4 in signed number = 1100 } already
 -2 in signed number = 1110 } 2's complement

∴ $-4 \div +2 = -2$
 $1100 \rightarrow 1110$

Registers Effected: SF, ZF, PF

AF: undefined

CF: last bit shifted out

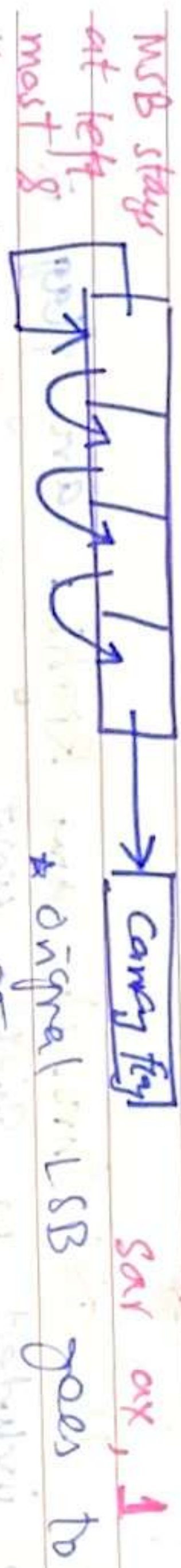
⇒ Rotate Right (lrcr): LSB goes to MSB as well as to carry flag.



ror ax, 1

Signed division by 2^n

⇒ Shift Arithmetic Right: To preserve the MSB which is signed bit & it won't affect the sign of number.



e.g: $1100 \xrightarrow{\text{goes to carry}} 1110$

-4 in signed number = 1100 } already
 -2 in signed number = 1110 } 2's complement

∴ $-4 \div +2 = -2$
 $1100 \rightarrow 1110$

Registers Effected: SF, ZF, PF

AF: undefined

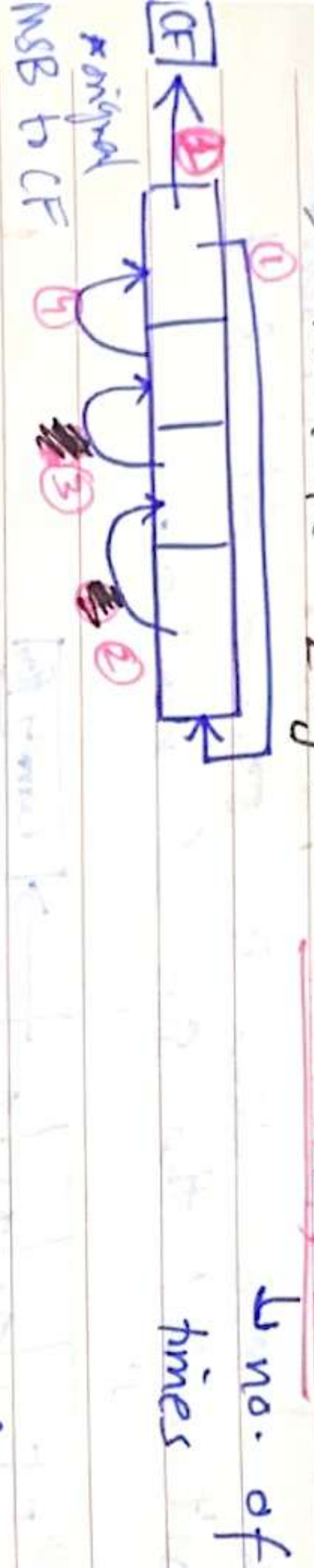
CF: last bit shifted out

⇒ Rotate Right (lrcr): LSB goes to MSB as well as to carry flag.

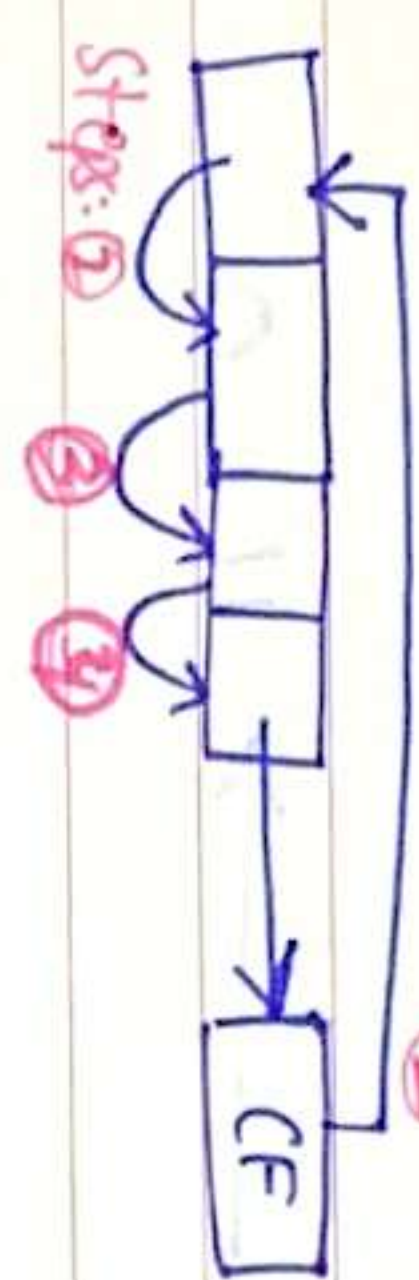


ror ax, 1

⇒ Rotate left: rol ax, 1

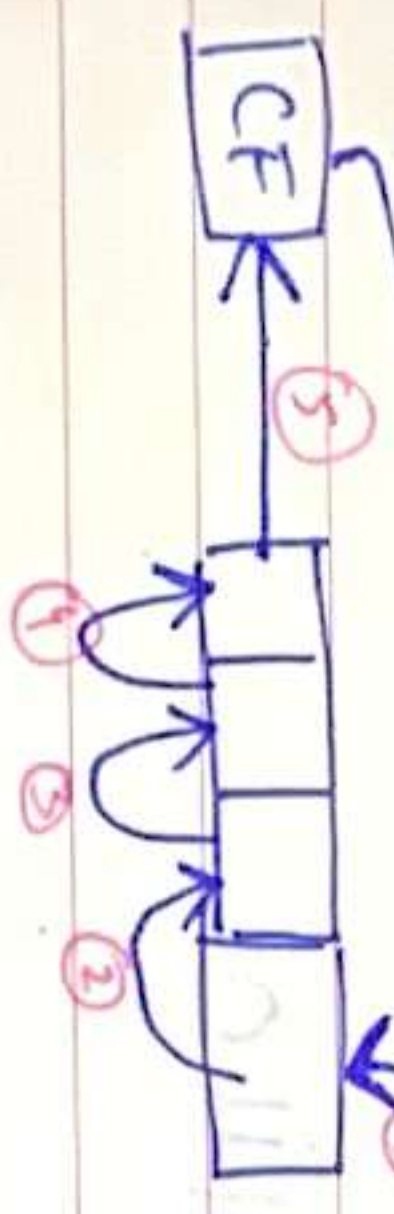


⇒ Rotate through Carry Right: rcr ax, 1



rcr ax, 1

⇒ Rotate through Carry left: rcl



Bitwise Logical Operation.

and al, bl

al	1	0	0	1	1	0	0
bl	1	0	1	0	1	1	0
al AND bl	1	0	0	0	1	0	0

∴ al = 10001000

or al, bl

Do bitwise or operation on bits of al & bl and store in al.

xor al, bl : bitwise xor on al & bl and store in al.

not al: Invert all 8 bits of 'al' (1's complement)

LECTURE-9

Tuesday 16/9/25

• Selective Bit Clearing: AND

1 0 1 0 1 → number
1 0 1 1 → mask

* Here we used a mask (1011) & AND it with our original number (10101) so we can clear 2nd Bit

from 1 → 0.
answer: 1001

Q: Clear 5th bit of al = 11100011

Mask: 11011111

11100011 AND 11011111 = 11000011

and al, 0x0F

Bit Clear: Make bit zero

Bit Set: Make bit 1

CLC and CARRY Flag C=0 and CARRY Flag C=1

STC Set Carry Flag C=1

CMC Invert/Complement CARRY Flag.

• Selective Bit Setting:

00001001: al
we need to set this bit

OR: 0100 mask

1101

OR al, 00001101

OR $X + 1 = 1$
 $X \cdot 0 = X$

• Selective Bit Invertor:

1001
invert this bit

XOR $X \oplus 0 = X$
 $X \oplus 1 = \bar{X}$

XOR: 0100

1100

• Selective Bit Testing: Checks if the specific bit is zero or 1 with help of zero flag.

test al, 1

$2^{n-1} = 2^{0-1} = 1$

al: 10100001

and 00000001

00000001

Non-Destructive AND operation
The result of AND won't be stored in register; it's checks

will make zero flag = 0;

The bit at 0th index is 1

clb

This is decimal / binary, we can only write a 2^n

number here since if we've to test 3rd bit,

we would have to write 1000 and AND it

with al since '1' is now at 3rd index

Overall formula:

test al, 2 (index-1)

for 4th bit = $2^{4-1} = 2^3$

test al, 8 OR test al, 0001000

⇒ 4 Bit Multiplication:

1101 → multiplicand (13)₁₀ ← bl
x 0101 → multiplier (5)₁₀ ← dl

1101

0000x

1101xx

0000xxx

[arg 0x0100]

jmp start

3. multiplier: dl 13

4. multiplier: dl 13 → result: dl 0

5. start:

6. mov cl, 4 // '4' is no. of bits remaining to be multiplied

7. mov bl, [multiplicand] 13

8. mov al, [multiplier]

9. check bit:

10. shr dl, 1 // goes to carry after 1 shr

11. jnc skip // checks carry bit. Carry will be 1

12. add [result], bl // if b/c of shift right, the bit that came out of right is 1; else 0.

13. skip

14. shl bl, 1

15. dec cl

16. jnz .checkBit

if b/c of shift right, the bit that came out of right is 1; else 0. In case of CF=0; answer of bl won't be added to result as it won't be effective as bl=0.

NOTE: For 8 Bit multiplication change line number:

③, ④

⑥: cl, 8

* Use bx & dx instead of bl & dl.

Num 1: $0x0050$

Num 2: $0x12345678$

4/2 Bits

Num 2: little endian	
78	least sig. byte
56	34
12	most sig. byte

⇒ Shifting 4 Byte no. Left:

Shl word [num 2], 1
 rcl word [num 2+2], 1

→ divided the 4 bytes no. into two 2 byte words.

① ↑ Shift towards left by 1 iteration.

5678 shl → 6780

0 goes to CF
 carry since 5 will be written like 0101 so left most 0 goes left to CF

② rcl will include the carry flag in shift operation:

CF 1
 2345 ← shift left 1234 [5]

Therefore answer = $0x5678ACF0$

As bits moved towards left not the digits.
 0100 → 1000 after shl.

mov ax, 0111b
 we can't write simple binary; we need to add the 'b' at end

- No spaces b/w binary

⇒ 64 bit addition: store number like "lower word, upper word" for a 0,2,4,6 sequence (else it could be 4,6,0,2).

LECTURE 10 Thursday 18/9/25

Extended

⇒ ↑ Shifting 4 Byte number to right: num1

num 1: dd 0x12345678

shr word [num 1+2], 1

rcl word [num 1], 1

So we can rotate right

The left most byte

we did '+2' since we defined word. Therefore, two 2 bytes will be jumped here (56 & 34).

1234 5678
 + 1234 5678

 Answer

⇒ Adding two 32 Bits No.

src	78	56	dest
src+1	56	34	dest+1
src+2	34	12	dest+2
src+3	12		dest+3

mov ax, [src] // 5678 wld go in ax

add [dest], ax // dest = 0x5678 + 0x5678

mov ax [src+2] // ax = 1234 (and CF not effected)

adc [dest+2], ax // 1234 + 1234 + CF

Extended.

⇒ Subtraction b/w two 4 Byte No.

mov ax, [src]

sub [dest], ax // If borrow needed; CF = 1

mov ax, [src+2] // If borrow needed; CF = 1

sbb [dest+2], ax // sbb will borrow & will give

subtract by 2

borrow 3-2-1 = 10-1 = 9

borrow 0 is also subtracted from left hand subtraction

⇒ Multiplication of 16 Bit:

multiplier: add → double the size of bits
 multiplier: du → to be multiplied (16 → 32)
 result: add

start: mov cl, 16 // counter
mov dx, [multiplier]

checkBit: shr dx, 1 // if carry = 1; then do addition
inc skip // shifting multiplier towards right.

mov ax, [multiplier]
add [result], ax
mov ax, [multiplier + 2] } Extended Addition
add [result + 2], ax

skip: shl word [multiplier], 1 } Extended
rcl word [multiplier + 2], 1 } shift left
dec cl

jnz checkbit

LECTURE - 11 Thursday 25/9/25.

SubRoutines / Functions

keyword Call Max → name of func.
int temp // not needed (just for understanding)
void main
ret

When 'max' func. will be called, the control would go to max. func where ever it is defined in the program. But at that point when line 1 is executed; the address of line 2 is stored in stack. Therefore when 'ret' keyword is called at the end of 'max' func; it knows that the control of handler has to go back to line 2 as the max func is fully executed now.

[org 0x100]

mov ax, 0

call increment

mov ax, 0x4C00

int 0x21

increment:

add ax, 1

How Ret knows where to go back in program when func. is finished? When 'increment' func. is called on line 3, the IP address of next line (line 4) is pushed in top of stack. Therefore when 'ret' is called on line 8; it fetches/pops that IP address from top of stack & goes back to execution from line 4 (where it left) when 'increment' was called.

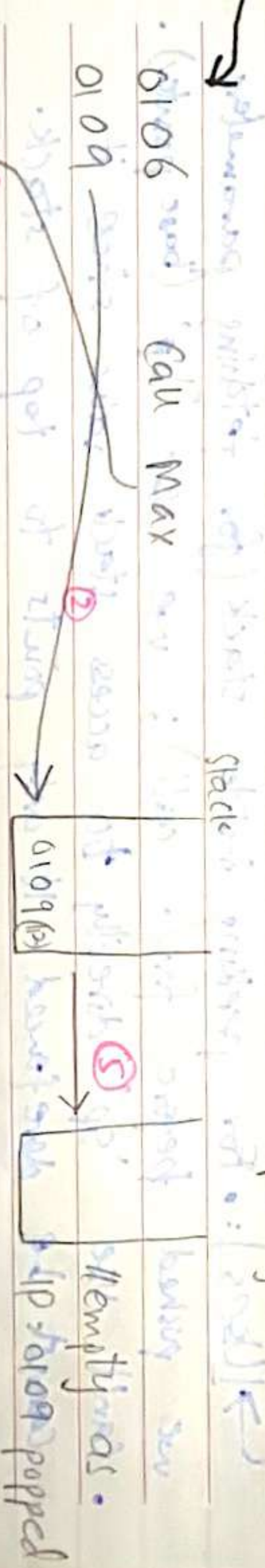
⇒ Stack & SP register on Databox:

We can only push or pop a word (2 Bytes) on stack. Therefore we'll always decrement/increment by '2'.
 SP

SP register contains the address of SP pointer (basically the pointer pointing to top of stack).

The default value of SP register is "FFFF" which will be index 'y' in above diagram. When data will be pushed; SP will change to FFFF - 2 = "FFFC".

Push: SP decrement by 2 (since we moved from 4th index to 2nd index & vice versa) while pushing value.



02FA
 Max: mov ax, 3
ret

IP address = 0100 + offset (003B in)

day / date:
 listing file

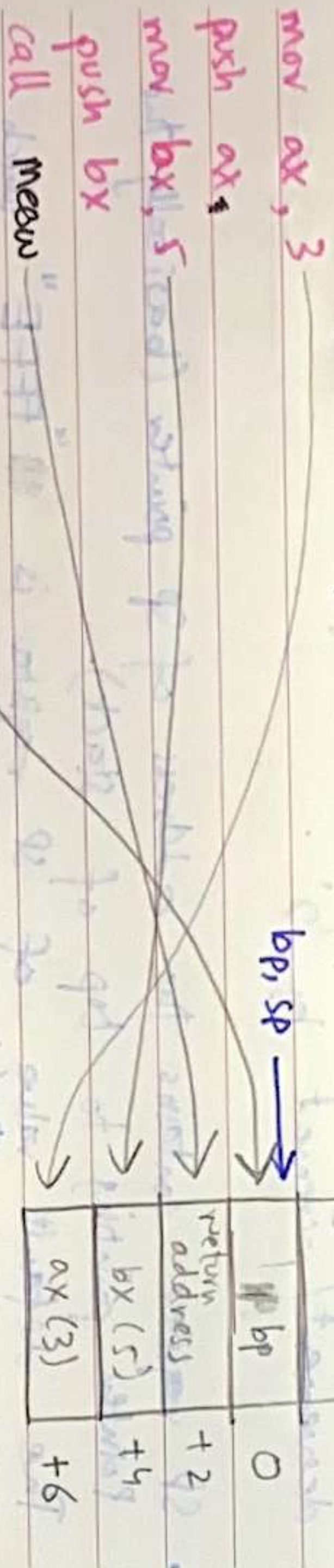
⇒ Stack Uses: Func. call & return

- Parameter's passing
- Saving & restoring registers
- Saving & return value
- Saving local variables

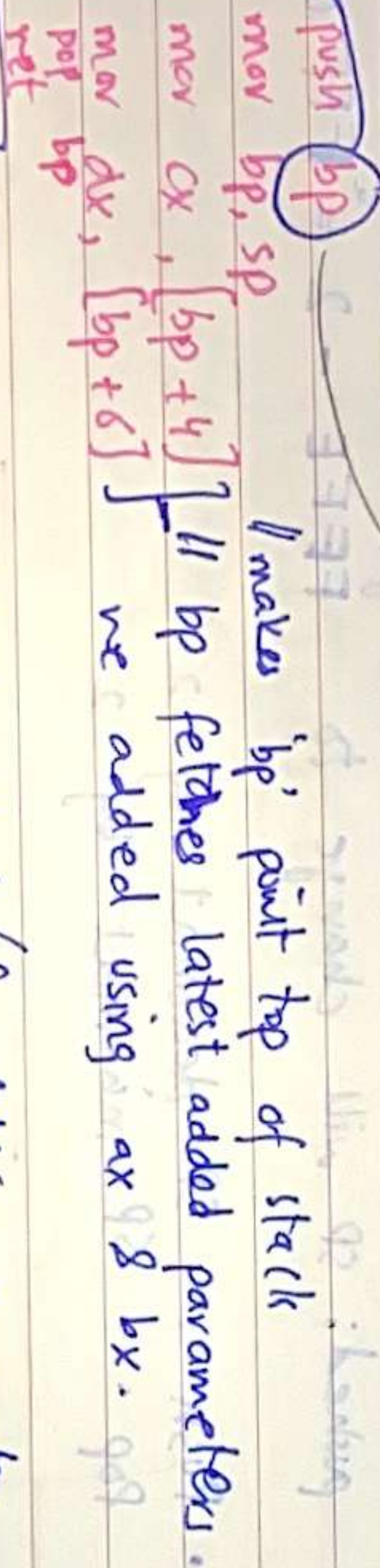
□ Passing Parameters in Subroutine using Stack.

- Registers could be used to pass parameters directly in Subroutines but since we're very limited registers; ∴ we use stack to pass parameters.

- Parameters are pushed in stack before calling the func. And then they're used in func. when it is called.



Meow:



- ↳ Uses: For peeking inside stack (for fetching parameters we pushed before func. call); we use 'bp' (base pointer). Can't use 'sp' directly to access stack value since it cannot be dereferenced. It only points to top of stack.

□ Push Pop Instructions:

push ax // value of ax → top of stack
 push word [num1] // value of num1 → " " } SP increments by 2.

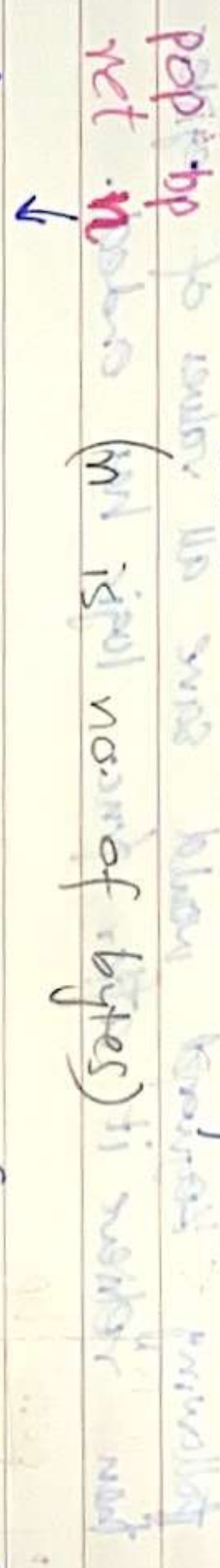
pop ax // copies value of top of stack to ax
 pop word [num1] // " " " " to num1

SP increments ∴ value is removed from top of stack.

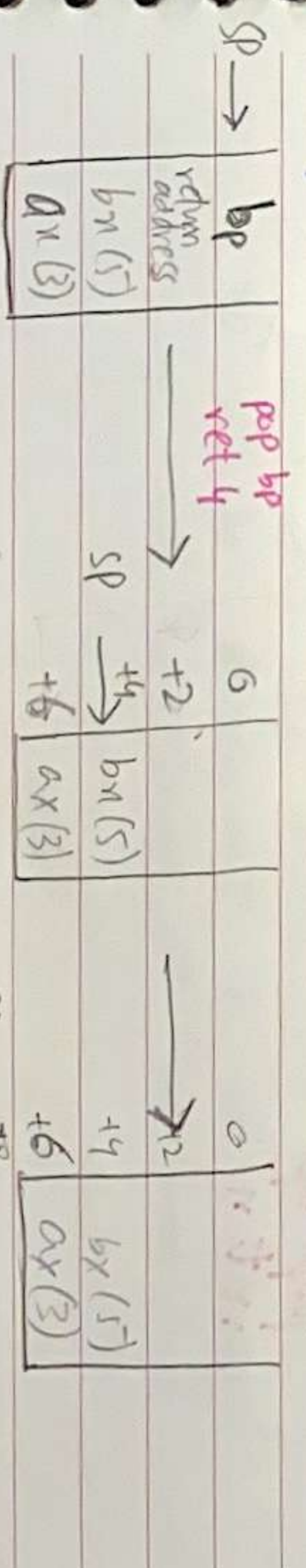
LECTURE-12 | Tuesday 30/9/25

⇒ Difference b/w "Jump" & "call func."

- Jump directly changes control to a new line
- Call func. also jumps to a new line but it also stores the IP address of next instruction in stack.



after handler gets back to main program (after leaving func.), the stack might have the parameters we passed before calling func. we need to clear that using "ret n" that will adjust the SP.



simple 'ret' does bes of ret 4. always pops 2 bytes which are 4+4=8 covered by return address.

push ax
call func1.

Creating local variable / date:

func1: push bp

mov bp, sp

mov from 4-6 sub sp, 2

after logic

mov sp, bp // losing the control

pop bp

ret 2.

of space for local variable" since handle is about to leave func. $\therefore$ local variables should be destroyed.

* To create two local variables, subtract '4' from sp and so on.

NOTE: For making space above bp (local variable), push bp first and then decrement sp.

For making space below bp (returning value), subtract sp first and then push bp.

Accessing local variable

func1: push bp

mov bp, sp

sub sp, 2

mov word [bp-2], 10

logic complete

mov sp, bp

pop bp

ret 2

local variable ko pop krna krna space krna. kahan hai hmtl, need to change sp.

Can't track sp. So we make bp as the reference. After sp will return, after ending with mov statement.

In nested func. (calling func. inside func), stack will still be there.

LECTURE-14

Tuesday
7/10/25

day / date:

ASCII: 7 bits $\rightarrow$ 2⁷ characters.

$$'A' - (65)_{10} = (0x41)_{16}$$

$$'a' - (97)_{10} = (0x61)_{16}$$

$\Rightarrow$ Extended ASCII: 8 bits $\rightarrow$ 2⁸ characters = 256

char1: db 'a' $\rightarrow$ 0x61
char2: db 'a', 'b', 'c' $\rightarrow$ 0x61, 0x62, 0x63
String 1: db 'abc' $\rightarrow$ 0x61, 0x62, 0x63

DISPLAY

80 columns

25 rows = 2000 characters.

Display in Dosbox

Each cell is of 2 byte (word) $\rightarrow$ 1st Byte: ASCII value $\rightarrow$ 2nd Byte: Attribute Byte

7 6 5 4 3 2 1 0

Background color intensity: colors of character

Blinking Bit: 1 $\rightarrow$ Blink on screen 1 $\rightarrow$ light/bright

0 $\rightarrow$ No Blink / Shil

LECTURE -15

Thursday
9/10/25

□ Display:

mov ax, 0xb800

mov es, ax

// 0xb800 is address of ↑ video memory.

// Loads video memory (ready to be used)

mov es, 0xb800 X

starting cell
we cannot transfer any
number directly into
segment register.

we're using segment register to access video memory since it's a separate part of RAM & cannot be accessed using normal registers (like: ax, bx etc) since they're only 16 bits whereas: 'es' segment register is of 64KB (larger memory for video).

mov word [es: 0], 0x0741

→ OFFSET

Attribute

07:

128 64 32 16 8 4 2 1

0 0 0 0 0 1 1 1

Bit R G B I R G B

= White not blinking character
on black background

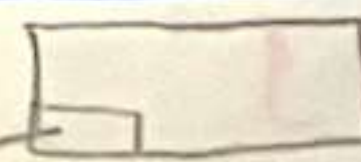
Data: $(41)_{16} = (65)_{10} = 'a'$

• "es: 0" tells at which coordinate (line & column) the letter we are typing will be displayed. The displaying starts from top left and moves towards right ∴ es: 0 = Extreme top left where the word 0x0741 will be displayed.

• If it was [es: 16], the letter would have been displayed in first column of

2nd row as one row = 80 words = 160 bits

• [es: 3840]: Last row, first column



0,0	0,1	0,2	...	0,79
1,0				
2,0				
...				
24,0				

Rows: 0 → 24 (0-24)
 Columns: 0 → 79 (0-79)



⇒ How to find OFFSET for specific row & column. (0-Based)

$$= ((\text{row} * 80) + (\text{column}) * 2)$$

Bottom right = 24th row, 79th column
 $((24 * 80) + 79) * 2 = 3998 \text{ bits}$

Center Screen = 12th row, 39th column
 $((12 * 80) + 39) * 2 = 1998 \text{ bits}$

⇒ MUL Command:

MUL BX // Multiplies ax and bx; and stores result in ax and dx
MUL BL // Multiplies al and bl; and stores result in al and ah (i.e: ax)

Note: for 'mul'; one operand is fixed (al or ax; depends upon either 2nd operand is 8 bit or 16bit). we just have to give another operand like bl, bx, etc.

⇒ Code for Calculating OFFSET:

```

mov bx, 0
mov al, 80
mov byte [rax] // row * 80
mov bl, [cal]
add ax, bx // (row * 80) + cols
shl ax, 1 // multiply by 2
mov di, ax
mov word [es:di], 0x0741
    
```

★ If we using 'loop' and cx; then max = 2000 (as it counts iterations)
 ★ If we using 'cmp di, 4000'; then max = 4000 (i.e: bytes)



Screen Clearing:

```

mov ax, 0xB800
mov es, ax
mov di, 0 // counter fill 2000 characters (full screen) filled.
nextChar:
mov word [es:di], 0x0720 // '20' = ASCII of space
add di, 2
cmp di, 4000 // see if full screen is covered with
jnc nextChar spaces.
    
```

String Printing:

```

message: db "Hello World"
mov di, 0 // offset
mov si, message // si would point to 'H' (first character of message)
mov cx, 11 // cx = length of string (including spaces)
mov ah, 0x07 // ah = attribute byte
nextChar:
mov al, [si] // moving ASCII code of character 'i.e: H' to 'al'
mov [es:di], al // displaying character in video memory (that is in 'es')
inc si // si points next character.
add di, 2 // offset points next memory location in video memory
loop nextChar // statement is equivalent to: dec cx
                                jnz nextChar.
    
```

To use 'loop', value of cx needs to be set before block of loop.

LECTURE - 16

Tuesday: ONLINE
14/10/25.

Number Printing (Display):

- Divide the number with 10
- The remainder will be LSB (rightmost digit)
- Convert that sliced digit into ASCII by adding 0x30 to the digit
- If the quotient is non-zero, repeat the process to slice next LSB.
- Pop digits one by one from stack and print from left to right.

Q. Display 59.

59/10 → Quotient = 5

Remainder = 9

9 + 0x30 = 0x39

push 0x39

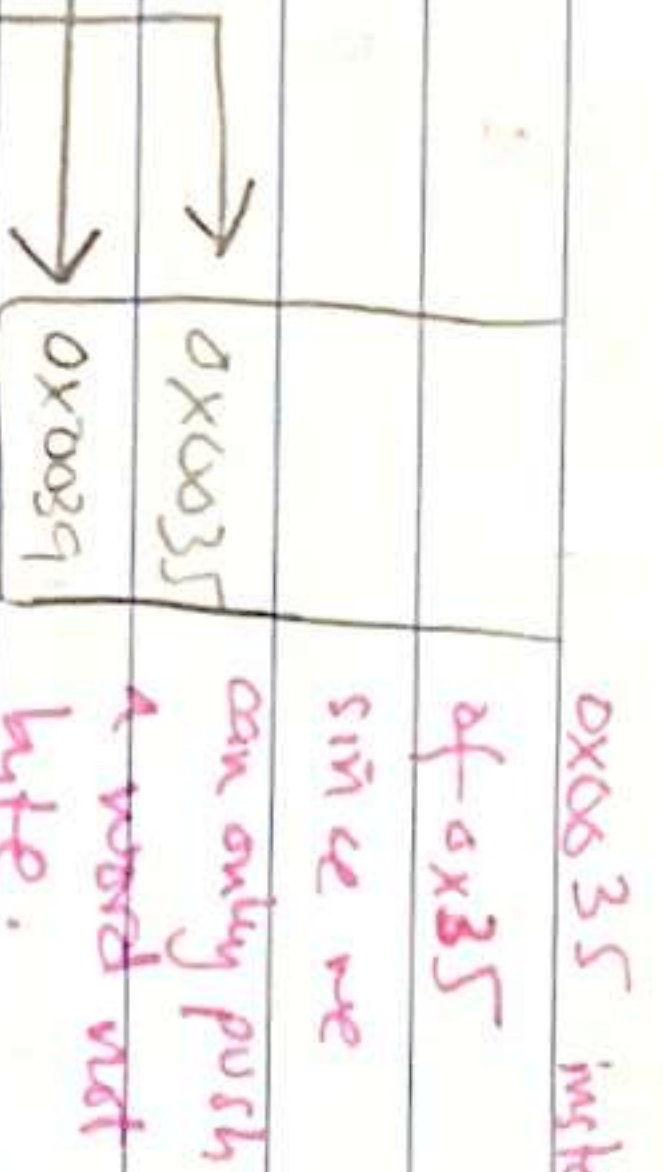
5/10 → Quotient = 0

Remainder = 5

5 + 0x30 = 0x35

push 0x35

Popping 0x0035 & then 0x0039 to display "59".



Divide Instruction:

- It also has one operand.
- Two forms:

(i) 32 bit / 16 bit

(ii) 16 bit / 8 bit.

- We can't know which flags are affected when div called.

(i) 32 bits → upper 16 bits in dx, lower in ax

Quotient: ax

Remainder: dx

div bx // dx ax
bx

(ii) 16 bits → stored in ax.

Quotient: ax

Remainder: ah

div bl // ax
bl

[NOTE]: It depends on the operand (bl or bx) which type of division will occur (32 bit or 16).

ES: DI: Video memory

DS: SI: Memory (like array list).



STRING INSTRUCTION:

CHAP # 7

- We can change multiple memory cells at the same time.
- Fixed destination & source operands
- If we use rep prefix, the instruction will be repeated cx times; and cx will be decremented after each repetition.

⇒ REP Prefix:

- It is like 'loop' beyond except it's a prefix not instruction which runs a statement cx times & decrements it to 0.
- The diff b/w rep & loop is that loop works for a block of code like an actual loop; whereas REP is just for a single instruction and nothing else can come while the REP iterations are being performed.

Rep STOS

- REP does not affect any flag.

NOTES: 'rep' doesn't work with CMPS and SCAS as it will only update flag with last value of string. we use 'repe' or 'repne'.

☐ CMPS: Compares two memory location like 2 strings

• CMPSB: for byte
CMPSW: for word

• DS: SI } SI and DI both
ES: DI } will be updated

repe } repeat while equal
repz } repeat while not equal

• Flags updated (like ZF)

⇒ Checking if two strings are equal (if same, BX=1)

STR1: DB 'ABCDEF'

STR2: DB 'ABCDEF'

MOV AX, DS

MOV ES, AX

MOV SI, STR1 // SI points to 'A' of STR1

MOV DI, STR2 // DI points to 'A' of STR2

MOV CX, 6

REPE CMPSB

JCXZ SAME

JMP END // CX goes from 6 to 0, means all characters match. The comparison broke in b/w somewhere as CX=0.

SAME: MOV BX, 1

END:



NOTE: for two string: 'ABDE' and 'ABCD'; the 'repe cmpsi' instruction breaks at CX=2 and moves to END BUT; SI and DI will be updated. 'SI' will point to 'D' (of STR1), 'DI' will point to 'C' (of STR2)

☐ MOV: Copy a block of memory to another block of memory.

• DS: SI → Source (STR1) } SI & DI will be
ES: DI → destination (STR2) } automatically updated.

⇒ Copy STR1 to STR2:

STR1: DB 'ABCDE'

STR2: DB 0,0,0,0,0 // '0' is ASCII of NULL.

MOV SI, STR1 // source to copy from

MOV DI, STR2 // destination to copied data.

MOV AX, DS // value of

MOV ES, AX

MOV CX, 5

REP MOVSB

LES ← Load data/segment. extra

When we push segments (ES, DS) and offsets (DI, SI) into stack; we can use LDS and LES instruction to do two things at once:

- 1) Moving segments into DS or ES LDS SI, [BP+6]
- 2) Move offsets into SI/DI. LDS SI, [BP+4]

of STR2.

could also be

LES DI, [BP+8] } for STR1

LES DI, [BP+10] }

bp	ret
offset 2	seg 2
offset 1	seg 1

GIVE Slides to API and
 ask it to explain everything in **LECTURE-17**

detail

CHAPTER: 8



Segmented Memory Model:

- Memory is partitioned in different segments like code, data, extra etc which is of 64KB.
- We can access memory using segment no. & offset no. (we've been just using offsets till now as we were in same segment so no seg no. required)
- offset No: is distance from start of segment to desired address.

Our code was in 2-3KB ∴ we need to switch segments.

TYPES OF JUMPS:

⇒ **Short Jump:** Conditional jumps are short jumps

- range: 128 bytes backward
- 127 bytes forward.

Relative single byte

⇒ **Near Jump:** Relative distance in 2 bytes

- Jump anywhere in segment. (seg size: 64KB)
- Segment same, offset diff.

jmp offset

jmp [01AD]

// shown while debugging.

will jump to this offset.

⇒ **Far Jump:** Can jump anywhere in memory (not just in seg).
 • Most like jmp, not relative.
 • Can jump b/w segments.
 • Need to provide both seg. & offsets.

jmp segment:offset → general syntax.
 jmp 1234h:5678h.

OR num: dd 0x12345678

jmp far [num]

↓ required

TYPES OF CALLS:

⇒ **Near Call:** All the calls of subroutines in prev. chap.

- Call anywhere in segment
- push return address on stack. (IP)
- 'ret' is used to return back to handler in main (pop IP).

⇒ **Far Call:**

- Call a Subroutine written in any segment.
- pushes both CS (code seg) & IP (return address) in stack.
- pop both CS & IP.
- retf

call segment: offset

call 1234h:5678h

INTERRUPT: Use to call temporary halt in program.

- It breaks the sequence of operation.
- Calling func. is also a type of interrupt.
- When interrupt call, handler goes to "Interrupt Service Routine" (ISR)
- IRET will give the handler back to where it left in main.

⇒ Internal Interrupts: Generated by within the program

1) Software Interrupts: Occurs when user writes an "int" instruction (like we write in end: int 0x21).

2) Processor Exception: Conditions like "divide overflow" or "divide error" that requires special handle.

⇒ External Hardware Interrupt: Caused by any peripheral device by sending signal. e.g: key pressed in keyboard will make an action.

near call: ip push
for call: ip & CS push
Extended Far Call: ~~for call: ip & CS push~~
Flags, CS and IP are pushed in Stack, in same order.

to save the current registers to be restored after interrupt ↑ is handled.

Input a character: ah = 01h, int 21h input a character what

ah = 0h, int 16h echo/outputting:

Output a string: ah = 9h, int 21h ah = 67h, int 21h

ah = 13h, int 10h

Output a character (in dl): ah = 02h, int 21h

⇒ 'int' instruction:

- Used inside program to request a "System Service" like program termination (int 0x21), user input, memory management, file access, disk access, display, set system date & time etc.

int <number> - where number lies b/w 0-255 or 0x0 - 0xFF
e.g: int 0x21

- Total interrupts: 256.

- parameters in these int func. are passed through registers (not stack) e.g: 01 transferred to ah first then int 21h called.
- Output is also returned in registers.

call → ret
int → iret.

System Services:

- ⇒ BIOS: service: Basic interface for hardware
- Processor comes in reset state when starts. CS is set to FFFFh & IP to 0000h. These addresses of segment & offset lets RAM transfer control to starting point of BIOS routines.

- BIOS loads OS.

⇒ Operating System Service: Perform I/O operations

- Manage memory & other resources
- e.g: DOS, windows, linux.
- Built upon BIOS services